FRCA Election App

A self-contained offline voting app for office bearer elections in Free Reformed Churches of Australia. Designed to run on a single laptop in the church hall with a local WiFi router. No internet required.

> **Not sure where to begin?** See where_to_start.md for a signpost that points you to the right document based on your role (council member, technical volunteer, etc.).

For other congregations considering this app

This is one congregation's interpretation of the Election Rules. Verify against your own.

The Rules for the Election of Office Bearers (Articles 1–13) leave room for interpretation on edge cases — for example, what counts as "valid votes cast" in Article 6a's threshold formula. This app encodes the maintaining congregation's reading, confirmed by their council. Other congregations may interpret some articles differently.

Before running an election with this software:

1. Read `voting-app/docs/ELECTION_RULES.md`. Each article appears verbatim alongside the app's interpretation and, where the call was non-obvious, the decision provenance.
2. Compare against your congregation's own Election Rules and council practice.
3. Where they differ, fork this repository and adjust `voting-app/election_rules.py` (the rule arithmetic) and the corresponding sites in `voting-app/app.py`. Tests in `voting-app/tests/test_rules_compliance.py` pin current behaviour and should be updated to match your interpretation.

The MIT License grants you the right to fork, modify, and run this software however suits your congregation. There is no obligation to upstream changes, though contributions are welcomed.

Before you start

This software was built for a specific congregation and is shared freely. It is provided as-is under the MIT License, without warranty of any kind.

A few things to keep in mind:

- **This is not a commercial product.** No help desk, no guaranteed support, no

service-level agreement. The author welcomes feedback but makes no commitment to act on it.

- **You are responsible for your own use.** Test thoroughly in your environment. Verify rule interpretations match your council's. Run a dry-run election before the real one.
- **Elections are serious.** Responsibility for running an election correctly rests with the church council, not with the authors of any tool used in the process.

This app works best when your congregation has someone comfortable with Python, basic networking, and the command line, willing to set things up, run a few dry runs, and be on hand on election day.

If you would prefer a fully supported solution, consider a commercial voting platform such as ElectionBuddy, or continue with traditional paper ballots. Both are perfectly good options.

What is this?

This app lets brothers cast ballots on their phones over a local WiFi network during congregational elections for elders and deacons. It runs on a single laptop. No internet connection required. Paper ballots remain available as a fallback at all times.

A typical election cycle:

1. Admin creates an election, enters candidate names, generates one-time voting codes.
2. Codes are printed on dual-sided cards (paper ballot on one side, code slip with QR on the other) and handed out after the chairman opens the election. Brothers sign the attendance register at the door.
3. Brothers vote on phone (scan QR or enter code) or on paper. If a brother doesn't have a phone, he can borrow one — the code slip is what identifies the vote, not the phone.
4. Each code can only be used once. Votes are recorded anonymously: no link between a code and the vote it cast.
5. Paper ballot tallies are entered manually after the round closes and added to digital totals.
6. Results display on a projector with full Article 6 threshold transparency. Multi-round elections carry candidates forward; per-round participants, ballots, and remaining vacancies are tracked automatically.

Quick start

Prerequisites: Python 3.11 or later.

```
cd frcdd_election_app/voting-app
pip install -r requirements.txt
```

Start the app using the launcher for your platform:

```
:: Windows
start.bat

:: Linux / macOS
./run.sh
```

The launcher starts the production server on port 5000 and opens the admin page in your browser. Do not run `python app.py` directly — it will print a reminder and exit.

The app runs on `http://localhost:5000`.

- Admin panel: `http://localhost:5000/admin`
- Voter page: `http://localhost:5000/`
- Projector display: `http://localhost:5000/display`

On first launch, the setup wizard collects your congregation name, WiFi SSID, and a new admin password. The default password is `admin`.

Election rules implementation

The app encodes Articles 1–13 of the Rules for the Election of Office Bearers. Highlights of what is and is not implemented:

Article	Implementation
2	Slate-size validation (2× vacancies) with Article 13 override.
4	Attendance register PDF generation; in-person count entry drives Article 6b.
5	Minutes DOCX export.
6a	**Reading A**: `valid_votes_cast` is the per-office sum of candidate ticks. Blanks and spoilts excluded. (See ELECTION_RULES.md decision log.)
6b	`ceil(participants × 2/5)`, fractions rounded up per Article 7.
7	Spoilt-ballot tracking per office; partial ballots accepted with warning; over-voting blocked on digital ballots. Subsequent ballots are chairman-driven.
10	Retiring-officer flag stored on candidates (informational).
11	Interim-election flag on elections; calendar-date checks skipped.
13	Slate-size override.

Procedural articles (1, 3, 8, 9, 12) are outside the app's scope — the chairman and council own those steps.

Full text and decision provenance: `voting-app/docs/ELECTION_RULES.md`.

Project layout

```
voting-app/
├── app.py                  # Flask application
├── election_rules.py       # Rule arithmetic (fork point for other
congregations)
├── templates/              # Jinja2 HTML templates
├── static/                 # CSS (all bundled, no CDNs)
├── data/                   # SQLite database (gitignored)
├── scripts/
│   ├── seed_demo.py        # Seed a demo election for dry runs
│   └── random_vote.py      # Cast random ballots (dry-run / load test)
├── tests/                  # pytest test suite
├── docs/
│   ├── ELECTION_RULES.md   # Verbatim rules + app interpretation +
provenance
│   ├── ADMIN_GUIDE.md      # How to run an election
│   ├── CONGREGATION_GUIDE.md  # What brothers see on election day
│   ├── SETUP.md            # Hardware / network setup
│   ├── SECURITY.md         # Security and anonymity model
│   └── FAILSAFE.md         # What to do when things go wrong
├── requirements.txt
```

```
└── LICENSE
LICENSE                     # Repo-root copy for GitHub auto-detect
CHANGELOG.md
```

Technology

Component	Technology
Backend	Python 3.11+, Flask
Database	SQLite (single file)
PDF generation	ReportLab
Production server	waitress
Frontend	HTML/CSS, mobile-first, no JavaScript required in the voter flow

Everything is self-contained. No CDNs, no internet dependencies, no external services. The voter flow works with JavaScript completely disabled — pure HTML forms on every phone from 2016 onwards.

License

MIT License. See LICENSE.

Built for office bearer elections in Free Reformed Churches of Australia. Released under the MIT License in the hope that it may serve any FRCA congregation, and any other organisation with similar needs.

Contact

Jan de Mooij

For ballot-sized printing of all materials in the printer pack, printer-admin@proecclesia.com.au is ready to support congregations that need it.